

學號: F112119106

姓名: 杜宜霖

Face Mask Detection with Tensorflow

應用情境: 過去在疫情期間口罩成為人人都必備的東西，進出任何場所都需要配戴口罩。即使現在疫情趨緩很多場合還是需要戴口罩才能進出，未來也不保證不會有新的疫情產生。

因此在 AI 普及的時代，可透過人工智慧來協助偵測人類是否有配戴口罩，減少接觸的機會。

資料集介紹: <https://www.kaggle.com/datasets/ashishjangra27/face-mask-12k-images-dataset/data>

此資料集用於影像的口罩偵測分類。資料集由近 12K 張影像組成，大小近 328.92MB。

所有戴口罩的圖像均從 Google 搜尋；所有無口罩的圖像均由 Jessica Li 創建的 CelebFace 資料集進行預處理 (<https://www.kaggle.com/jessicali9530>)。

Loading Necessary Imports

```
[ ] import os
import random
import seaborn as sns
from sklearn.metrics import confusion_matrix
import numpy as np
import pandas as pd
import tensorflow as tf

import matplotlib.pyplot as plt
plt.rcParams['figure.figsize'] = (12, 8)
from IPython import display
import shutil

print("Version: ", tf.__version__)
print("GPU is", "available" if tf.config.list_physical_devices('GPU') else "NOT AVAILABLE")

Version: 2.13.0
GPU is NOT AVAILABLE
```

Downloading Dataset 下載資料集

```
[ ] # installing kaggle and uploading api key to download dataset directly in colab 直接從kaggle下載
!pip install -q kaggle
from google.colab import files
files.upload()
```

選擇檔案 未選擇任何檔案

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
Saving kaggle.json to kaggle.json
{'kaggle.json': b'{"username": "elainedu10", "key": "c5df4609e823cdb792abe635a26bdf1"}'}
```

```
▶ # making a directory named kaggle and save api key json file there
! mkdir ~/.kaggle
! cp kaggle.json ~/.kaggle
```

```
[ ] # changing permission of the json file
! chmod 600 ~/.kaggle/kaggle.json
```

```
[ ] # verifying if I am able to access kaggle
! kaggle datasets list
```

```
# verifying if I am able to access kaggle
! kaggle datasets list
```

ref	title	size	lastUpdated	downloadCount	voteCount	usabilityRating
iamsouravbanerjee/customer-shopping-trends-dataset	Customer Shopping Trends Dataset	146KB	2023-10-05 06:45:37	8465	194	1.0
nelgiriwithana/top-spotify-songs-2023	Most Streamed Spotify Songs 2023	47KB	2023-08-26 11:04:57	36993	1084	1.0
nelgiriwithana/billionaires-statistics-dataset	Billionaires Statistics Dataset (2023)	139KB	2023-09-29 13:39:28	4536	121	1.0
bkcoban/customer-transactions	Customer Transactions	1MB	2023-10-15 17:59:17	918	23	1.0
nelgiriwithana/credit-card-fraud-detection-dataset-2023	Credit Card Fraud Detection Dataset 2023	143MB	2023-09-18 10:00:19	6965	224	1.0
anshtanwar/monthly-food-price-estimates	Global Food Price Inflation	254KB	2023-10-21 15:33:25	2714	63	1.0
yakhyojon/tiktok	TikTok User Engagement Data	794KB	2023-10-18 01:21:10	707	27	1.0
redpen12/mobile-phone-market-in-ghana	Mobile Phone Market in Ghana	49KB	2023-10-18 10:33:10	380	29	1.0
muhammadawaistayyab/amazon-movies-and-films	Amazon- Movies and Films	92KB	2023-10-15 01:42:04	1084	39	1.0
rajatsurana979/hotel-reservation-data-repository	Hotel Reservation Data Repository	4MB	2023-10-17 04:02:28	293	29	1.0
iamsouravbanerjee/heart-attack-prediction-dataset	Heart Attack Risk Prediction Dataset	519KB	2023-09-27 07:07:50	3783	84	1.0
iamsouravbanerjee/cars-dataset	Cars Dataset	571KB	2023-10-17 05:58:29	806	30	1.0
willianoliveiragabin/10000-data-about-movies-1915-2023	10,000 Data about movies (1915 - 2023)	1MB	2023-10-11 19:31:12	1492	40	1.0
sujoykapadnis/spotify-songs	spotify songs	3MB	2023-10-17 09:17:20	626	23	1.0
teamincirbo/cyber-security-attacks	Cyber Security Attacks	5MB	2023-10-14 14:12:52	926	30	0.9411765
asaniczka/wages-by-education-in-the-usa-1973-2022	Wages by Education in the USA (1973-2022)	7KB	2023-10-09 09:43:40	1736	49	1.0
sujoykapadnis/salaries-by-gender	Salaries by gender	44KB	2023-10-11 14:06:43	502	62	1.0
willianoliveiragabin/fatalities-in-the-israeli-palestinian	Fatalities in the Israeli-Palestinian	463KB	2023-10-10 19:02:21	1068	57	1.0
amohankumar/global-nutrition-report-dataset	Global Nutrition Report Dataset	127KB	2023-10-18 05:33:00	272	22	1.0
asaniczka/amazon-uk-products-dataset-2023	Amazon UK Products Dataset 2023 (2.2M Products)	131MB	2023-10-15 18:00:15	551	31	1.0

```
# downloadig the particular dataset using the api command from the dataset page 下載資料集
!kaggle datasets download -d ashishjangra27/face-mask-12k-images-dataset
```

Downloading face-mask-12k-images-dataset.zip to /content

95% 313M/330M [00:03<00:00, 77.7MB/s]

100% 330M/330M [00:03<00:00, 94.6MB/s]

```
# unzipping the data 解壓縮檔案
```

```
import zipfile
zip_ref = zipfile.ZipFile('face-mask-12k-images-dataset.zip', 'r')
zip_ref.extractall('/content')
zip_ref.close()
```

```
#掛載至雲端硬碟
```

```
from google.colab import drive
drive.mount('/content/drive')
```

Configurations

```
▶ PATH = os.getcwd() #返回當前工作目錄
DATA_PATH = PATH + '/Face Mask Dataset'

LR = 0.001
BATCH_SIZE = 16
EPOCHS = 32
TARGET_SIZE = (256, 256)
NUM_CLASSES = 2
```

Utility Functions

```
[ ] # function to count parameters of a model計算模型參數

def count_params(model):
    non_trainable_params = np.sum([np.prod(v.get_shape().as_list())
                                     for v in model.non_trainable_weights])
    trainable_params = np.sum([np.prod(v.get_shape().as_list())
                                for v in model.trainable_weights])
    print(non_trainable_params, trainable_params)
    return {'non_trainable_params': non_trainable_params, 'trainable_params': trainable_params}
```

```
[ ] # function to save figures儲存圖表

def save_fig(fig_id, tight_layout=True, fig_extension="png", resolution=300):
    os.chdir(PATH)
    IMAGES_PATH = PATH + '/images'

    if os.path.isdir(IMAGES_PATH) is False:
        os.mkdir('images')
        IMAGES_PATH = PATH + '/images'
    path = os.path.join(IMAGES_PATH, fig_id + "." + fig_extension)
    print("Saving figure", fig_id)

    if tight_layout:
        plt.tight_layout()
    plt.savefig(path, format=fig_extension, dpi=resolution)
    return
```

```
) # function for plotting training訓練圖
```

```
def plot_training(history, name):  
    hist = pd.DataFrame()  
    hist["Train Loss"]=history.history['loss']  
    hist["Validation Loss"]=history.history['val_loss']  
    hist["Train Accuracy"]=history.history['accuracy']  
    hist["Validation Accuracy"]=history.history['val_accuracy']  
  
    fig, axarr=plt.subplots(nrows=2, ncols=1 ,figsize=(8,6))  
    axarr[0].set_title("History of Loss in Train and Validation Datasets")  
    hist[["Train Loss", "Validation Loss"]].plot(ax=axarr[0])  
    axarr[1].set_title("History of Accuracy in Train and Validation Datasets")  
    hist[["Train Accuracy", "Validation Accuracy"]].plot(ax=axarr[1])  
    save_fig(name)  
    plt.show()
```

```
# plotting the outputs using confusion matrix混淆矩陣繪製輸出
```

```
def plot_confusion(y_test,y_predict,name):  
    cm = confusion_matrix(y_test, y_predict)  
    class_names=[0,1,2,3,4,5,6,7,8,9]  
    fig, ax = plt.subplots(figsize=(6,6))  
    tick_marks = np.arange(len(class_names))  
    plt.xticks(tick_marks, class_names)  
    plt.yticks(tick_marks, class_names)  
    sns.heatmap(pd.DataFrame(cm), annot=True, cmap="Purples" ,fmt='g')  
    ax.xaxis.set_label_position("top")  
    plt.tight_layout()  
    plt.title('Confusion Matrix', y=1.1)  
    plt.ylabel('Actual label')  
    plt.xlabel('Predicted label')  
    save_fig(name)  
    plt.show()  
    return
```

```
# plotting images from train dir從Training目錄繪製影像

def plot_images(PATH):
    images = []
    for folder in os.listdir(PATH):
        for image in os.listdir(PATH + '/' + folder):
            images.append((os.path.join(PATH, folder, image), folder))

    plt.figure(1, figsize=(8, 8))

    for i in range(16):
        i += 1

        random_img = random.choice(images)
        imgs = plt.imread(random_img[0])
        plt.subplot(4, 4, i)
        plt.axis('off')
        plt.title(random_img[1])
        plt.imshow(imgs)
    save_fig('train_imgs')
    plt.show()

plot_images(DATA_PATH + '/Train')
```

Saving figure train_imgs



```
# image distribution影像分佈

def count_plot(path):
    class_counts = {}

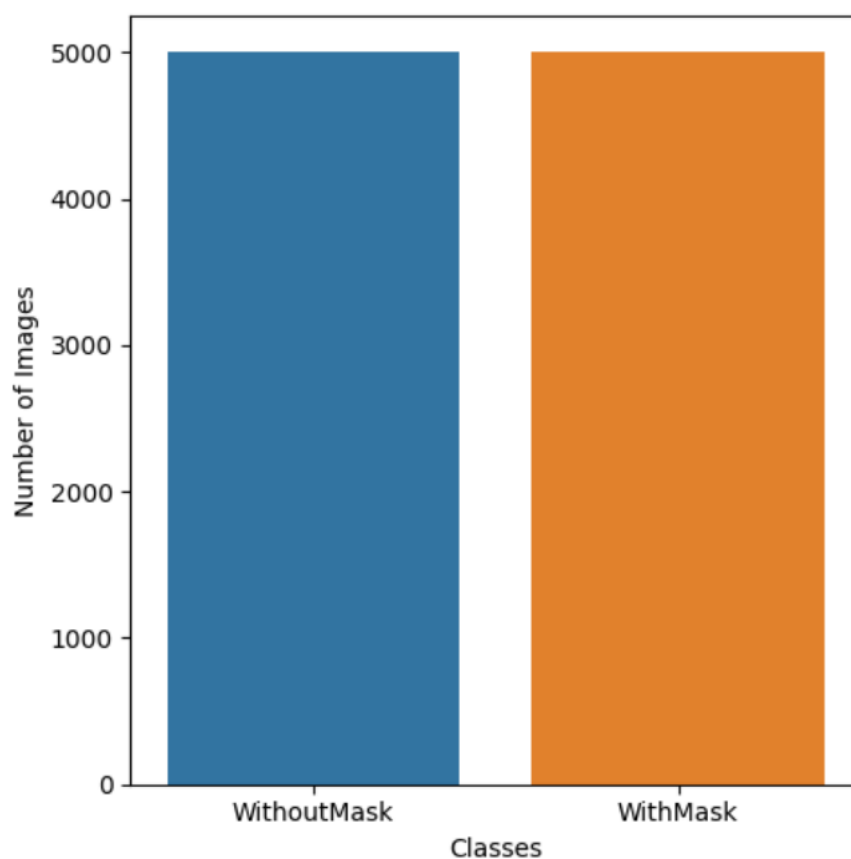
    for subdir in os.listdir(path):
        subdir_path = os.path.join(path, subdir)
        class_counts[subdir] = len(os.listdir(subdir_path))

    fig = plt.figure(figsize=(5, 5))

    sns.barplot(x=list(class_counts.keys()), y=list(class_counts.values()), data=None)
    plt.xlabel('Classes')
    plt.ylabel('Number of Images')
    save_fig('count_plt')
    plt.show()

count_plot(DATA_PATH + '/Train')
```

Saving figure count_plt



```
# pie chart to view image distribution visually 圓餅圖

def pie_chart(path):
    os.chdir(path)
    list_dir = []
    list_dir = os.listdir(os.getcwd())
    count = {}
    for dir in list_dir:
        file = os.listdir(os.getcwd()+ '/' +dir)
        count[dir] = len(file)

    fig = plt.figure(figsize=(5, 5))

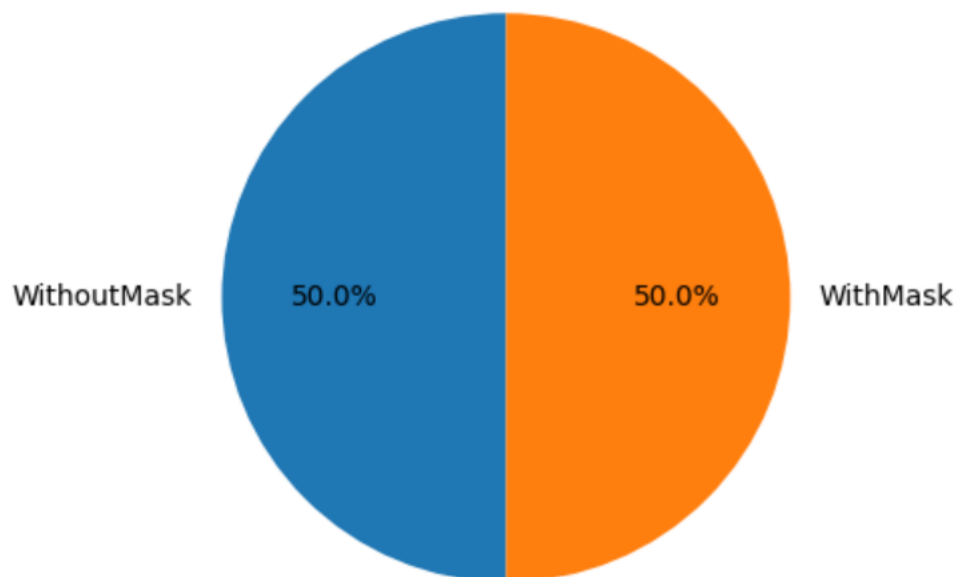
    plt.pie(list(count.values()), labels=list(count.keys()), autopct='%1.1f%%',
            startangle=90.0, radius=15.0)

    plt.axis('equal')
    plt.title("Pie Chart")
    save_fig('pie_chart')
    plt.show()
    return

pie_chart(DATA_PATH + '/Train')
# Validation Data also has almost same distriibution but test data is uniform
```

Saving figure pie_chart

Pie Chart



```

# function to create test, train and valid batches

from keras.preprocessing.image import ImageDataGenerator

def create_batches(path):
    train_path = path + '/Train'
    test_path = path + '/Test'
    valid_path = path + '/Validation'

    # Create a data augmentor 影像增强
    data_augmentor = ImageDataGenerator([

        samplewise_center=True,
        rescale=1./255,
        shear_range=0.2,
        zoom_range = 0.2,
        samplewise_std_normalization=True])

    train_batches = data_augmentor.flow_from_directory(

        directory=train_path,
        target_size=TARGET_SIZE,
        batch_size=BATCH_SIZE,
        class_mode='binary')

    test_batches = data_augmentor.flow_from_directory(

        directory=test_path,
        target_size=TARGET_SIZE,
        shuffle=False,
        batch_size=BATCH_SIZE,
        class_mode='binary')

    valid_batches = data_augmentor.flow_from_directory(

        directory=valid_path,
        target_size=TARGET_SIZE,
        batch_size=BATCH_SIZE,
        class_mode='binary')

    return (train_batches, test_batches, valid_batches)

PATH = DATA_PATH
train_batches, test_batches, valid_batches = create_batches(PATH)

```

Found 10000 images belonging to 2 classes.
Found 992 images belonging to 2 classes.
Found 800 images belonging to 2 classes.

Model Creation 建造模型

```
[ ] # function for model creation

from keras.layers import Conv2D, MaxPooling2D, Dropout, Dense, Flatten
from keras.models import Sequential
def create_model():

    model = Sequential()
    #第1層卷積 feature map 卷積核的數目16個 kernel 卷積核大小3*3
    model.add(Conv2D(16, (3, 3), activation = "relu", input_shape=(256, 256, 3)))
    model.add(MaxPooling2D(pool_size=(2, 2)))
    model.add(Dropout(0.2))
    #第2層卷積
    model.add(Conv2D(32, (3, 3), activation = "relu"))
    model.add(MaxPooling2D(pool_size=(2, 2)))
    model.add(Dropout(0.2))
    model.add(Conv2D(32, (3, 3), activation = "relu"))
    model.add(MaxPooling2D(pool_size=(2, 2)))
    model.add(Dropout(0.2))
    model.add(Conv2D(64, (3, 3), activation = "relu"))
    model.add(MaxPooling2D(pool_size=(2, 2)))
    model.add(Dropout(0.2))
    model.add(Flatten()) #推平成一行
    model.add(Dense(128, activation = "relu")) #Dense全連接層 128個神經元
    model.add(Dropout(0.2))
    model.add(Dense(1, activation = "sigmoid"))

    model.summary()

    return model
```

```
[ ] model = create_model()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 254, 254, 16)	448
max_pooling2d (MaxPooling2D)	(None, 127, 127, 16)	0
dropout (Dropout)	(None, 127, 127, 16)	0
conv2d_1 (Conv2D)	(None, 125, 125, 32)	4640
max_pooling2d_1 (MaxPooling2D)	(None, 62, 62, 32)	0
dropout_1 (Dropout)	(None, 62, 62, 32)	0
conv2d_2 (Conv2D)	(None, 60, 60, 32)	9248

Model Training訓練模型

```
[ ] def train_and_evaluate_model(model):

    model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=0.0001),
                  loss=tf.losses.BinaryCrossentropy(),
                  metrics=[tf.metrics.BinaryAccuracy(name='accuracy')])

    history = model.fit(train_batches,
                        epochs=EPOCHS,
                        batch_size=BATCH_SIZE,
                        validation_data=valid_batches,
                        callbacks=[tf.keras.callbacks.EarlyStopping(monitor='val_loss', patience=3, mode='min')],
                        verbose=1)

    return (history, model)
```

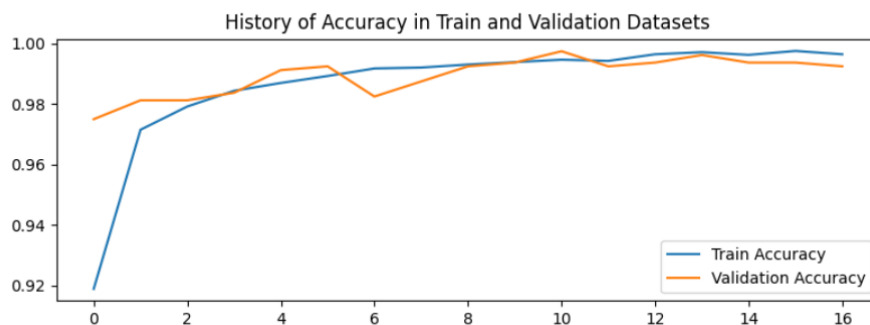
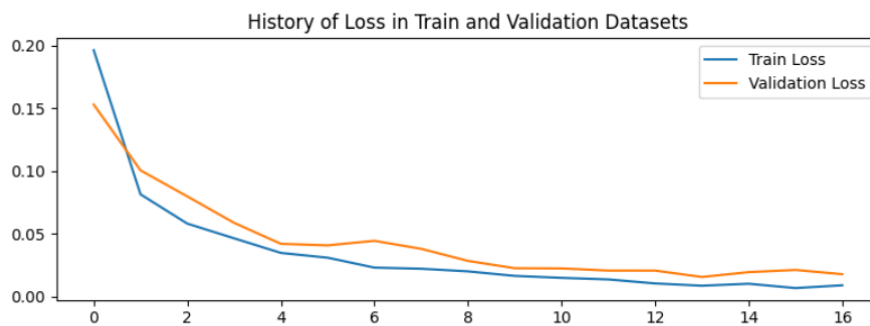
```
history, trained_model = train_and_evaluate_model(model)
```

```
Epoch 1/32
625/625 [-----] - 441s 699ms/step - loss: 0.1963 - accuracy: 0.9188 - val_loss: 0.1530 - val_accuracy: 0.9750
Epoch 2/32
625/625 [-----] - 430s 687ms/step - loss: 0.0816 - accuracy: 0.9715 - val_loss: 0.1006 - val_accuracy: 0.9812
Epoch 3/32
625/625 [-----] - 427s 683ms/step - loss: 0.0582 - accuracy: 0.9792 - val_loss: 0.0799 - val_accuracy: 0.9812
Epoch 4/32
625/625 [-----] - 425s 680ms/step - loss: 0.0465 - accuracy: 0.9844 - val_loss: 0.0589 - val_accuracy: 0.9837
Epoch 5/32
625/625 [-----] - 438s 700ms/step - loss: 0.0349 - accuracy: 0.9870 - val_loss: 0.0422 - val_accuracy: 0.9912
Epoch 6/32
625/625 [-----] - 431s 690ms/step - loss: 0.0311 - accuracy: 0.9893 - val_loss: 0.0409 - val_accuracy: 0.9925
Epoch 7/32
625/625 [-----] - 439s 702ms/step - loss: 0.0232 - accuracy: 0.9918 - val_loss: 0.0445 - val_accuracy: 0.9825
Epoch 8/32
625/625 [-----] - 491s 785ms/step - loss: 0.0223 - accuracy: 0.9921 - val_loss: 0.0382 - val_accuracy: 0.9875
Epoch 9/32
625/625 [-----] - 449s 717ms/step - loss: 0.0202 - accuracy: 0.9931 - val_loss: 0.0286 - val_accuracy: 0.9925
Epoch 10/32
625/625 [-----] - 444s 710ms/step - loss: 0.0166 - accuracy: 0.9939 - val_loss: 0.0227 - val_accuracy: 0.9937
Epoch 11/32
625/625 [-----] - 438s 700ms/step - loss: 0.0151 - accuracy: 0.9947 - val_loss: 0.0226 - val_accuracy: 0.9975
Epoch 12/32
625/625 [-----] - 443s 709ms/step - loss: 0.0138 - accuracy: 0.9943 - val_loss: 0.0208 - val_accuracy: 0.9925
Epoch 13/32
625/625 [-----] - 445s 711ms/step - loss: 0.0106 - accuracy: 0.9965 - val_loss: 0.0208 - val_accuracy: 0.9937
Epoch 14/32
625/625 [-----] - 438s 700ms/step - loss: 0.0088 - accuracy: 0.9972 - val_loss: 0.0158 - val_accuracy: 0.9962
Epoch 15/32
```

展示訓練成果

```
plot_training(history, "base_conv_train")
```

```
Saving figure base_conv_train
```

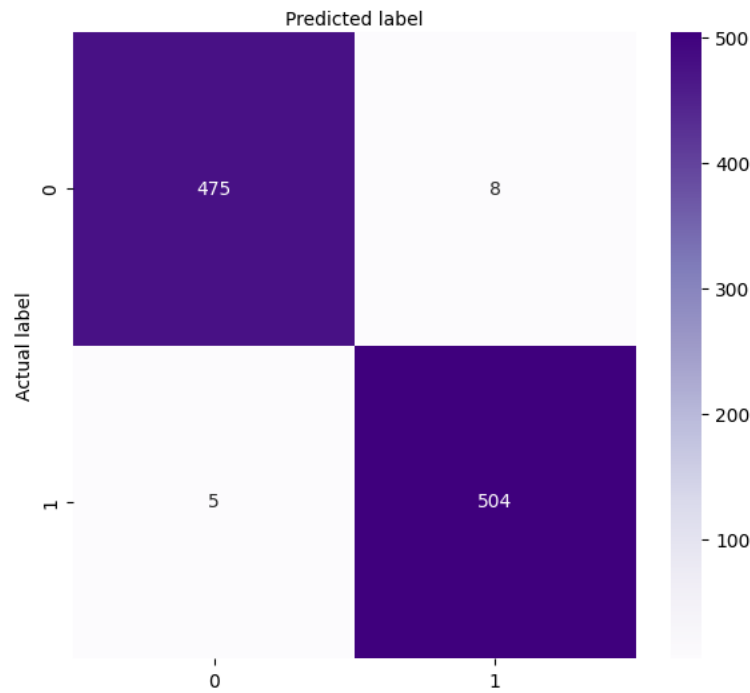


Model Testing

```
test_labels = test_batches.classes
predictions = trained_model.predict(test_batches)
y_pred= np.round(predictions)
plot_confusion(test_labels,y_pred, 'base_conv')
```

62/62 [=====] - 20s 315ms/step
Saving figure base_conv

Confusion Matrix



Saving Model

```
# saving model
M_PATH = PATH + '/models'
model.save(M_PATH + "/base_model.h5")
```

/usr/local/lib/python3.10/dist-packages/keras/src/engine/training.py:3000: UserWarning: You are saving your model as an HDF5 file via `model.save()`. This file

saving_api.save_model()

```
[30] from keras.models import load_model
def mload_model():
    model = tf.keras.models.load_model('/content/Face Mask Dataset/models/base_model.h5', compile=False)
    return model
```

```
[31] model = mload_model()
```

Making Predictions

On Image

```
1 2 from matplotlib.layout_engine import nullcontext
3 from tensorflow.python.framework.ops import NullContextManager
4 import cv2
5 mask_label = ['0: WITH MASK', 1: 'NO MASK']
6 shape_color = {0: (255, 0, 0), 1: (0, 0, 255)}
7 text_color = {0: (255, 0, 0), 1: (0, 0, 255)}
8 #trying it out on a sample image
9 face_model = cv2.CascadeClassifier(cv2.data.haarcascades + 'haarcascade_frontalface_default.xml')
10 img = cv2.imread('/content/images/1007.png')
11
12 img = cv2.cvtColor(img, cv2.IMREAD_GRAYSCALE)
13 faces = face_model.detectMultiScale(img, scaleFactor=1.1, minNeighbors=1) #returns a list of (x,y,w,h) tuples
14 if len(faces) == 0:
15     print("未检测到人脸")
16 new_img = cv2.cvtColor(img, cv2.COLOR_RGB2BGR) #colored output image
17
18 #plotting
19 for (x,y,w,h) in faces:
20     crop = new_img[y:y+h, x:x+w]
21     crop = cv2.resize(crop, (256, 256))
22     crop = np.reshape(crop, [1, 256, 256, 3])/255.0
23     mask_result = (model.predict(crop)).flatten().tolist()
24     mask_result1 = round(mask_result[0])
25     print(mask_label[mask_result1])
26
27     cv2.putText(new_img, mask_label[mask_result1], (x, y-10), cv2.FONT_HERSHEY_SIMPLEX, 0.5, text_color[mask_result1], 2)
28     cv2.rectangle(new_img, (x, y), (x+w, y+h), shape_color[mask_result1], 1)
29 plt.figure(figsize=(10,10))
30
31 plt.axis('off')
32 save_fig('mask_detection2')
33 plt.imshow(new_img)
```

```
1/1 [=====] - 0s 39ms/step
WITH MASK
Saving figure mask_detection2
<matplotlib.image.AxesImage at 0x7e6aa8e6ce0>
```



參考資料:<https://github.com/tajammulbasheer/face-mask-detection/tree/main#model>
<https://www.kaggle.com/datasets/ashishjangra27/face-mask-12k-images-dataset/data>